[Hung Nguyen] Process Fundamentals, Memory Layout



Before reading this document, we assume a grasp of the basics and will focus only on interesting or novel insights, as well as unforeseen situations.



1.Process Identifier

`fork()`

Use `fork()` to create a child process that has a copy of certain elements from the parent.
Example:

```
#include <stdio.h>
#include <unistd.h>

int main(void)
{
    printf("Before fork\n");
    printf("PID = %d\n", getpid());   /* pid of current process */
    printf("PPID = %d\n", getppid()); /* pid of parent process */
    printf("-------------------------------------------------\n");

    pid_t pid = fork();

    printf("After fork\n");
```

```
    if (pid < 0) /* error */
    {
        perror("fork");
        return -1;
    }

    if (pid == 0) /* child process */
    {
        printf("Child:  PID = %d\n", getpid());
        printf("Child: PPID = %d\n", getppid());
    }
    else /* parrent process */
    {
        printf("Parent: PID = %d\n", getpid());
        printf("Parent: child PID = %d\n", pid);
    }
    printf("-------------------------------------------------\n");

    return 0;
}
```

```
Before fork
PID = 52228
PPID = 21052
-------------------------------------------------
After fork
Parent: PID = 52228
Parent: child PID = 52229
-------------------------------------------------
After fork
Child:  PID = 52229
Child: PPID = 52228
-------------------------------------------------
```

`fork()` creates a child process corresponding to a parent process and continues execution from the point immediately following the `fork()` call:

```
       P
       |
```

```
     fork()
     /    \
    P      C
```

What happens if `fork()` is called twice?

```
fork();
fork();
```

```
       P
      / \
     P   C
    / \ / \
   P  C C  C
```

We get 4 processes.
If `fork()` is called 3 times, it will create $2^3 = 8$ processes.
Similarly, N consecutive calls to `fork()` will create `2^N` processes.

Another example when using `fork()`:

```
#include <stdio.h>
#include <unistd.h>

int main(void)
{
    printf("Hello");

    fork();

    return 0;
}
```

You might also be interested in:

```
Hello
```

but you may see that:

```
Hello
Hello
```

Why?
Before fork():

```
stdio buffer
     │
     └── "Hello"

If data is stil in the buffer:

              fork()
                │
          ┌─────┴─────┐
          │           │
     Parent          Child
          │           │
     buffer          buffer
     "Hello"         "Hello"
```

Both processes have a copy of the buffer.

Then, both terminaled normally:

```
Parent → flush
Child  → flush
```

→ "Hello" may appears twice.

If we use "Hello\n", stdout will be flushed immediately before calling `fork()`, so the result will show "Hello" once.

```
#include <stdio.h>
#include <unistd.h>

int main(void)
{
    printf("Hello\n");

    fork();

    return 0;
}
```

```
Hello
```

vfork()

`vfork` also creates a child process, just like `fork`; however, unlike `fork`, the child process created by `vfork` temporarily shares its memory space with the parent process. Consequently, the parent process is suspended until the child process calls `exit()` or `_exit()`.

what happen if i forget to call `exit()` in the child process? Following this examle below:

```
int main(void)
{
    int x = 20;

    printf("hello\n"); /* print "hello" before vfork()*/

    pid_t pid = vfork(); /* create child 1 in parent process */

    if (pid == 0) /* in child 1 process */
    {
        printf("child1 x = %d\n", x);

        pid_t pid1 = vfork(); /* create child 2 in child 1 process */

        if (pid1 == 0) /* in child 2 process */
        {
            printf("child2\n");
            // exit(12); /* i comment it */
        }
        else /* in child 1 process */
        {
            // exit(0); /* i comment it */
        }
    }
    else /* in parent process */
    {
        printf("Parent : %d\n", x);
    }

    printf("parrent + child : x = %d\n", x);
```

```
    return 24;
}
```

```
hello
child1 x = 20
child2
parrent + child : x = 20
parrent + child : x = 32492
Parent : 32492
parrent + child : x = 32492
```

We can observe the program running normally until child process 2 executes the statement `printf("child2\n");` (I have commented out the `exit()` command so it does not execute). Subsequently, the program proceeds to execute `printf("parrent + child : x = %d\n", x);` without any intervening `exit()` call. Similarly, child process 1 also executes `printf("parrent + child : x = %d\n", x);`, though the value of `x` is undefined. Once child process 1 terminates, the parent process resumes execution. Finally, the program outputs an unexpected value for `x`.



This example serves to demonstrate that failing to call `exit()` within `vfork()` can have a significant impact on data shared between the parent and child processes. In practice, the use of nested `vfork()` calls is generally discouraged.

`wait()`

If the parrent creates a child:

```
pid_t pid = fork();
```

then the parrent can wait for the child:

```
wait(NULL);
```

Note that wait() will wait for any child process to terminate first; if two child processes terminate simultaneously, it will randomly select one and return its PID.

Example when using `wait()`:

- If you **don't use** `wait()`, the parent process terminates before the child process, therefore the child process is adopted by the system via an appropriate process.

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    pid_t pid = fork();

    if (pid == 0)
    {
        sleep(0.5);
        printf("Child:  PID = %d\n", getpid());
        printf("Child: PPID = %d\n", getppid());
        printf("Child finished\n");
        printf("-----------------------------------------------------\n");
    }
    else
    {
        printf("Parent: PID = %d\n", getpid());
        printf("Parent: child PID = %d\n", pid);
        printf("-----------------------------------------------------\n");
        /* don't use wait() */
        // wait(NULL);
        printf("Parent finished\n");
    }

    return 0;
}
```

```
Parent: PID = 95959
Parent: child PID = 95960
-----------------------------------------------------
Parent finished
Child:  PID = 95960
Child: PPID = 2109
Child finished
-----------------------------------------------------
```

- If you **use** `wait()`, the parent process will wait for the child process to finish before terminating itself.

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    pid_t pid = fork();

    if (pid == 0)
    {
        sleep(0.5);
        printf("Child:  PID = %d\n", getpid());
        printf("Child: PPID = %d\n", getppid());
        printf("Child finished\n");
        printf("-----------------------------------------------------\n");
    }
    else
    {
        printf("Parent: PID = %d\n", getpid());
        printf("Parent: child PID = %d\n", pid);
        printf("-----------------------------------------------------\n");
        /* use wait() */
        wait(NULL);
        printf("Parent finished\n");
    }

    return 0;
}
```

```
Parent: PID = 98725
Parent: child PID = 98726
-----------------------------------------------------
Child:  PID = 98726
Child: PPID = 98725
Child finished
-----------------------------------------------------
Parent finished
```

- If you want to wait the specific child process, you must use `waitpid()`. Basically, it works just like `wait()`, with one difference: it waits specifically for the process with the

designated PID to finish running.

Zombie process

The child process terminates before the parent process calls `wait()`. Child becones **Zombie**. It is not an active process and does not consume CPU or memory but it can still occupy an entry in the process table.

When a process becomes a zombie, it releases a large amount of resources but retains a small amount of information regarding:

```
PID
termination status
resource usage information
parent information
```

The purpose is to allow the parent to call `wait()`.

Having one or a few zombie processes typically does not significantly impact the OS; however, a large number of zombie processes can be problematic, as each one still requires a corresponding process table entry or task structure within the kernel—and, crucially, a PID.

Linux has a limit on the number of PIDs/processes. We can view this:

```
cat /proc/sys/kernel/pid_max
```

```
4194304
```

In addition, each user/process also has resource limits:

```
ulimit -u
```

```
30090
```

If a program continuously creates child processes without calling `wait()`, the following sequence can eventually occur:

```
Process/PID resources
↓
exhausted
↓
fork() fails
```

```
↓
system/program cannot create further processes
```

For example, if I create 10,000 processes and turn them into zombies, resources will eventually be exhausted, preventing me from creating any new processes.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void)
{
    printf("Parent PID = %d\n", getpid());

    /* create 10000 process and turn them to zombie */
    for (int i = 0; i < 10000; i++) {

        pid_t pid = fork();

        if (pid < 0) {
            perror("fork");
            break;
        }

        if (pid == 0) {
            /* Child exit immediately */
            exit(0);
        }

        printf("Created child %d PID = %d\n", i + 1, pid);
    }

    printf("Parent sleeping...\n");
    getchar();

    return 0;
}
```

```
Created child 9019 PID = 61802
Created child 9020 PID = 61803
```

```
Created child 9021 PID = 61804
Created child 9022 PID = 61805
Created child 9023 PID = 61806
Created child 9024 PID = 61807
Created child 9025 PID = 61808
fork: Resource temporarily unavailable
Parent sleeping...
```

As you can see, I was able to create 9025 processes; however, at process 9026, the `fork` operation reported an error—*Resource temporarily unavailable*—due to a lack of resources.

We can see processes by using:

```
ps -eo pid,ppid,pgid,sid,stat,cmd | grep Z
```

Zombies have a status field of Z+.

```
  61802   52783   52783   10229 Z+   [process] <defunct>
  61803   52783   52783   10229 Z+   [process] <defunct>
  61804   52783   52783   10229 Z+   [process] <defunct>
  61805   52783   52783   10229 Z+   [process] <defunct>
  61806   52783   52783   10229 Z+   [process] <defunct>
  61807   52783   52783   10229 Z+   [process] <defunct>
  61808   52783   52783   10229 Z+   [process] <defunct>
```

Orphan process

The parent process terminates before the child process terminates. This is similar to the example **don't use** `wait()` above.

Example:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void)
{
    pid_t pid = fork();

    if (pid == 0)
```

```
    {
        /* child will sleep to wait parrent die */
        sleep(0.2);

        printf("Child PID  = %d\n", getpid());
        printf("Child PPID = %d\n", getppid());

        return 0;
    }
    else
    {
        printf("Parent PID  = %d\n", getpid());
        printf("Child PID = %d\n", pid);
    }

    printf("Parent exiting...\n");

    exit(0);
}
```

```
Parrent:
------------------------------
Parent PID  = 69237
Child PID = 69238
Parent exiting...
Child:
------------------------------
Before parrent die:
--------
Child PID  = 69238
Child PPID = 69237
After parrent die:
--------
Child PID  = 69238
Child PPID = 2176
```

Everything proceeds normally while the parent process is alive; however, when the parent dies, the child process becomes an orphan and is adopted by another process.
Typically, the child process is adopted by the process with PID 1 (usually systemd). However, this is not always the case; Linux features a re-parenting mechanism that allows a process to register a request:

"*If any of my descendants become orphans, re-parent them to me before assigning them to PID 1.*"

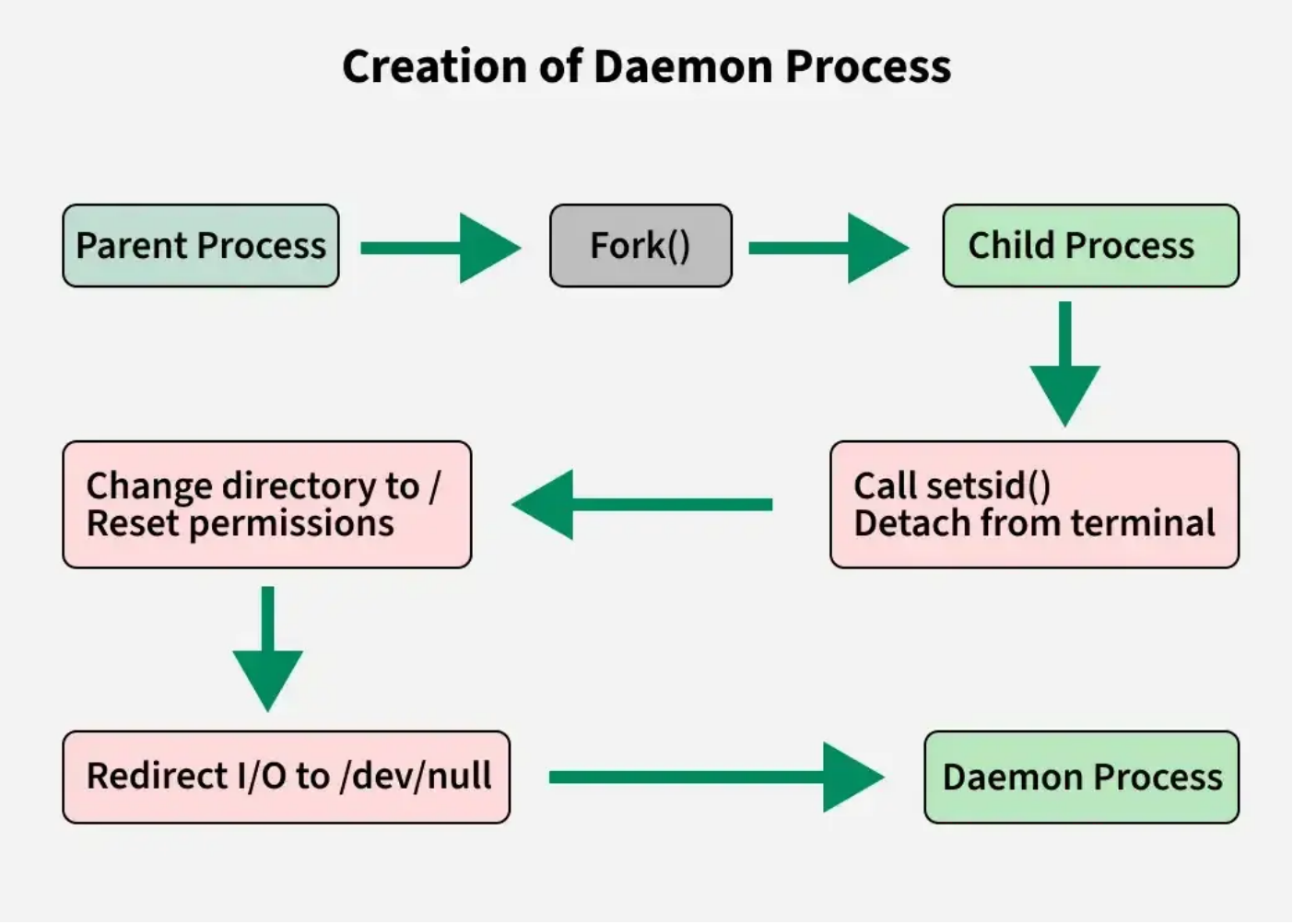
Consequently, we often see child processes being adopted by a process with a PID other than 1.

Daemon process

"*A daemon process is a background process that runs independently of any user control and performs specific tasks for the system. Daemons are usually started when the system starts, and they run until the system stops. A daemon process typically performs system services and is available at all times to more than one task or user. Daemon processes are started by the root user or root shell and can be stopped only by the root user.*"

A daemon is a background process that typically does not interact directly with the user's terminal.

We can create a Daemon proces by following steps below:



Use this command to view process:

```
ps -ef
```

We can see too many daemon process that have status field of "?":

```
root           1       0  0 08:23 ?        00:00:01 /sbin/init splash
root           2       0  0 08:23 ?        00:00:00 [kthreadd]
root           3       2  0 08:23 ?        00:00:00 [pool_workqueue_release]
root           4       2  0 08:23 ?        00:00:00 [kworker/R-rcu_g]
root           5       2  0 08:23 ?        00:00:00 [kworker/R-rcu_p]
root           6       2  0 08:23 ?        00:00:00 [kworker/R-slub_]
root           7       2  0 08:23 ?        00:00:00 [kworker/R-netns]
root           9       2  0 08:23 ?        00:00:00 [kworker/0:0H-events_highpri]
root          12       2  0 08:23 ?        00:00:00 [kworker/R-mm_pe]
root          13       2  0 08:23 ?        00:00:00 [rcu_tasks_kthread]
root          14       2  0 08:23 ?        00:00:00 [rcu_tasks_rude_kthread]
root          15       2  0 08:23 ?        00:00:00 [rcu_tasks_trace_kthread]
```

Why we need Daemon instead of process?

For example, if I want to write an program to control a robot, I need to create processes for reading sensors, processing data, handling events, controlling motors, and so on. I do not want to have to open multiple terminal windows and keep them running every time I operate the robot. That is where daemons come in handy; I simply start the robot, and the processes run automatically without needing a terminal

Example create a simple daemon:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>

/*
    function print ids of preocess:
        PID
        PPID
        PGID
        SID
        TID
*/
void print_id(const char *name)
{
    printf("\n--- %s ---\n", name);
    printf("PID  = %d\n", getpid());
    printf("PPID = %d\n", getppid());
```

```
    printf("PGID = %d\n", getpgid(0));
    printf("SID  = %d\n", getsid(0));
    printf("TID  = %ld\n", (long)getpid());
}

int main(void)
{
    print_id("BEFORE FORK");

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (pid > 0)
    {
        /* Parent exits */
        printf("\nParent exits. Parent PID = %d\n", getpid());
        exit(EXIT_SUCCESS);
    }

    /* Child */
    print_id("AFTER FORK");

    /* Create a new session */
    if (setsid() == -1)
    {
        perror("setsid");
        exit(EXIT_FAILURE);
    }

    print_id("AFTER SETSID");

    /*
        Daemon normally does not need a working directory
        tied to the user's current directory.
    */
```

```
    chdir("/");

    /*
        Close standard file descriptors
        In this example, I want to see that daemon is still running
        so I will commnad it.
    */
    // close(STDIN_FILENO);
    // close(STDOUT_FILENO);
    // close(STDERR_FILENO);

    /* child become daemon and run forever */
    while (1)
    {
        sleep(5);

        printf("\nDaemon is still running...\n");
        print_id("DAEMON");
    }

    return 0;
}
```

```
--- BEFORE FORK ---
PID  = 92736
PPID = 10229
PGID = 92736
SID  = 10229
TID  = 92736

Parent exits. Parent PID = 92736

--- AFTER FORK ---
PID  = 92737
PPID = 92736
PGID = 92736
SID  = 10229
TID  = 92737
```

```
--- AFTER SETSID ---
PID  = 92737
PPID = 2176
PGID = 92737
SID  = 92737
TID  = 92737

Daemon is still running...

--- DAEMON ---
PID  = 92737
PPID = 2176
PGID = 92737
SID  = 92737
TID  = 92737

Daemon is still running...

--- DAEMON ---
PID  = 92737
PPID = 2176
PGID = 92737
SID  = 92737
TID  = 92737
```

2. Race condition betwen fork() and exec()



To recap briefly on `exec()`: it replaces the current process with a new program but it does not create a new process; consequently, the PID and parent-child relationships remain unchanged. An example of a race condition between a parent process and a child process when both access the memory area pointed to by the `counter` pointer.

```
#include <stdio.h>
/* library for mmap() */
```

```
#include <sys/mman.h>
#include <stdlib.h>
/* library for fork() */
#include <unistd.h>

int main(int argc, char *argv[])
{
    int *counter = mmap(NULL,
                        sizeof(int),
                        PROT_READ | PROT_WRITE,
                        MAP_SHARED | MAP_ANONYMOUS,
                        -1,
                        0);

    *counter = 0;

    pid_t pid = fork();

    /* error fork */
    if(pid < 0)
    {
        perror("fork");
        exit(-1);
    }

    for(int i = 0; i < 100000; i++)
    {
        (*counter)++;
    }

    if(pid > 0)
    {
        printf("Final counter = %d\n", *counter);
    }

    return 0;
}
```

```
Final counter = 94990
```

The result I expected is 200000, but the result I actualy receive is 94990. Two processes race to read from and write to the same shared resource.

Another example, if a child process is writing to a file but then calls `exec()`, the child's original data is completely lost and replaced by the new program.

```
int main()
{
    pid_t pid = fork();

    if (pid == 0)
    {
        /* CHILD */
        printf("Child: preparing important data...\n");

        FILE *fp = fopen("important.txt", "w");
        fprintf(fp, "IMPORTANT DATA\n");

        printf("Child: now exec another program...\n");

        /* call exec() */
        execl(".../test", "test", NULL);

        perror("exec");
        exit(127);
    }
    else
    {
        wait(NULL);
        printf("I am parent.  Child already done!\n");
    }
}
```

```
Child: preparing important data...
Child: now exec another program...
I am new program.  Goodbyte!
I am parent.  Child already done!
```

3.Compare system() and execvp()

As we know, `system()` essentially requests the shell to run an executable while the calling process continues its own work, whereas `execvp()` replaces the current program with a new one without creating a separate process.

☹️

When you write:

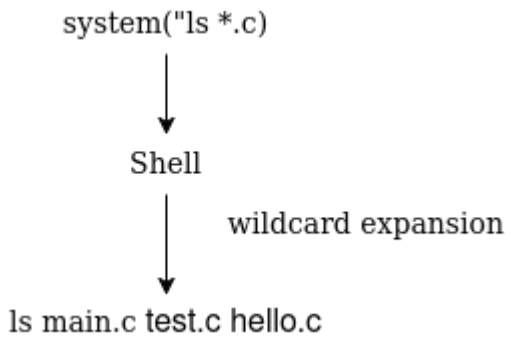
```
system("ls *.c");
```

The shell interprets:

```
*.c
```

and expands it. For example, if your directory contains the files main.c, test.c, and hello.c, the command will output:

```
main.c
test.c
hello.c
```



So `ls` effectively receives:

```
ls main.c test.c hello.c
```

This expansion is called **pathname expansion** (often called globbing).

With `execvp()`:

```
char *args[] = {
    "ls",
    "*.c",
    NULL
};

execvp("ls", args);
```

there is no shell doing wildcard expansion.

`ls` receives:

```
"*.c"
```

as an argument.

This distinction is extremely important.

The `system()` function executes a command via the shell, which interprets characters like `|`, `>`, `<`, `*`, `&&`, and `;` to carry out the command; in contrast, `execvp()` lacks a shell parser, so it treats those characters as part of an ordinary string.

One important warning

Don't blindly do this:

```
char command[100];

sprintf(command, "ls %s", user_input);

system(command);
```

If user_input comes from an untrusted user, they could potentially inject shell commands.

For example, conceptually:

```
user_input = "abc; rm ..."
```

Then your program could accidentally execute more than intended.

This is called command injection.
For system programming, this is one reason direct exec*() interfaces are often preferable when you already know the executable and arguments.

4 Process Groups, Sessions, Controlling Terminal & Job Control

Process group

The Open Group define process group: "*A collection of processes that permits the signaling of related processes. Each process in the system is a member of a process group that is identified by a process group ID. A newly created process joins the process group of its creator.*"

- An simple example about the child process use `setpgid()` to become new process group:

```
int main(void)
{
    printf("parent:\n");
    printf("PID  = %d\n", getpid());
    printf("PPID: %d\n", getppid());
    printf("PGID = %d\n", getpgrp());
    printf("--------------------------------\n");

    pid_t pid = fork();

    if (pid == 0)
    {
        printf("child:\n");
        printf("child PID  = %d\n", getpid());
        printf("child PPID: %d\n", getppid());
        printf("child PGID = %d\n", getpgrp());
        printf("--------------------------------\n");

        setpgid(0, 0);
        pid_t pid = fork();

        if (pid == 0)
```

```
        {
            while (1)
            {
                printf("grandchild:\n");
                printf("grandchild PID  = %d\n", getpid());
                printf("grandchild PPID: %d\n", getppid());
                printf("grandchild PGID = %d\n", getpgrp());
                printf("------------------------------\n");
                sleep(1);
            }
        }
        else
        {
            while (1)
            {
                printf("child:\n");
                printf("child PID  = %d\n", getpid());
                printf("child PPID: %d\n", getppid());
                printf("child PGID = %d\n", getpgrp());
                printf("------------------------------\n");
                sleep(1);
            }
        }
    }
    else
    {
        sleep(7);
        kill(-pid, SIGTERM); /* sent signal SIGTERM to process group */
    }

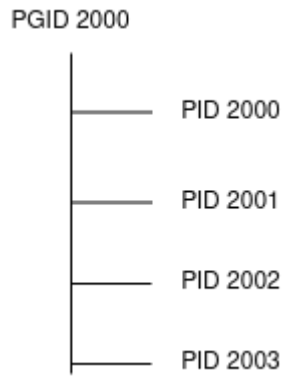
    return 0;
}
```

```
parent:
PID  = 72705
PPID: 8443
PGID = 72705
------------------------------
child:
child PID  = 72706
```

```
child PPID: 72705
child PGID = 72705
--------------------------------
child:                                  <---------- after setpgid()
child PID  = 72706
child PPID: 72705
child PGID = 72706
--------------------------------
grandchild:
grandchild PID  = 72707
grandchild PPID: 72706
grandchild PGID = 72706
--------------------------------
```

The child process 2, created by child process 1, also have the same process group id as the child process 1.
When parent execute the command `kill(pgid, SIGTERM);`, the signal is sent not only to a single process but also to all members of that process group.

Example:
We have a group:



Then we execute the command `kill(-2000, SIGTERM)`, Kernel will send SIGTERM to all members of the process group.

Session



A process belongs to only one process group, and a process group belongs to only one session.

When the process call `setsid()`, if it meets the conditions, it becomes: session leader and process-group leader. At the same time, it detaches from the current controlling terminal. This forms the

foundation of a **Daemon process**.

- For example:

```
int main(void)
{
    printf("Before setsid:\n");
    printf("PID  = %d\n", getpid());
    printf("PPID = %d\n", getppid());
    printf("PGID = %d\n", getpgrp());
    printf("SID  = %d\n", getsid(0));

    printf("--------------------------------\n");

    pid_t pid = fork();

    if (pid == 0)
    {
        printf("Child before setsid:\n");
        printf("PID  = %d\n", getpid());
        printf("PPID = %d\n", getppid());
        printf("PGID = %d\n", getpgrp());
        printf("SID  = %d\n", getsid(0));

        printf("--------------------------------\n");

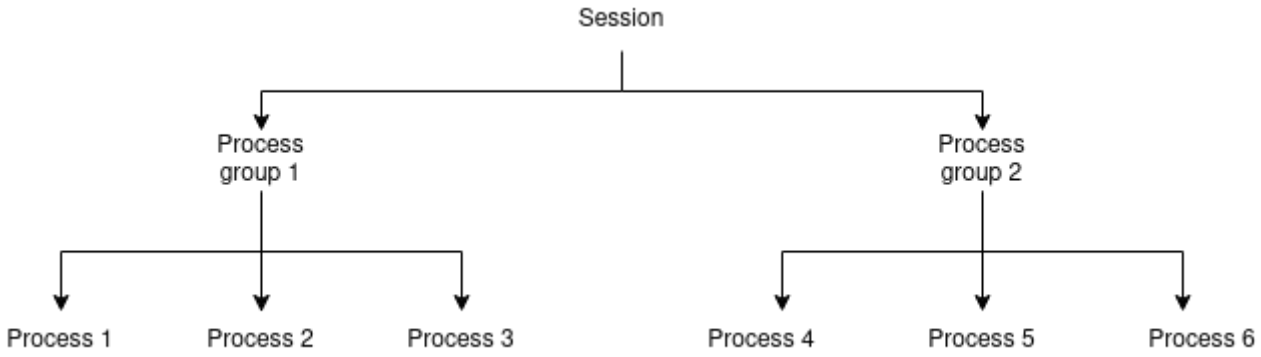
        setsid();

        printf("Child after setsid:\n");
        printf("PID  = %d\n", getpid());
        printf("PPID = %d\n", getppid());
        printf("PGID = %d\n", getpgrp());
        printf("SID  = %d\n", getsid(0));
    }
    sleep(1);

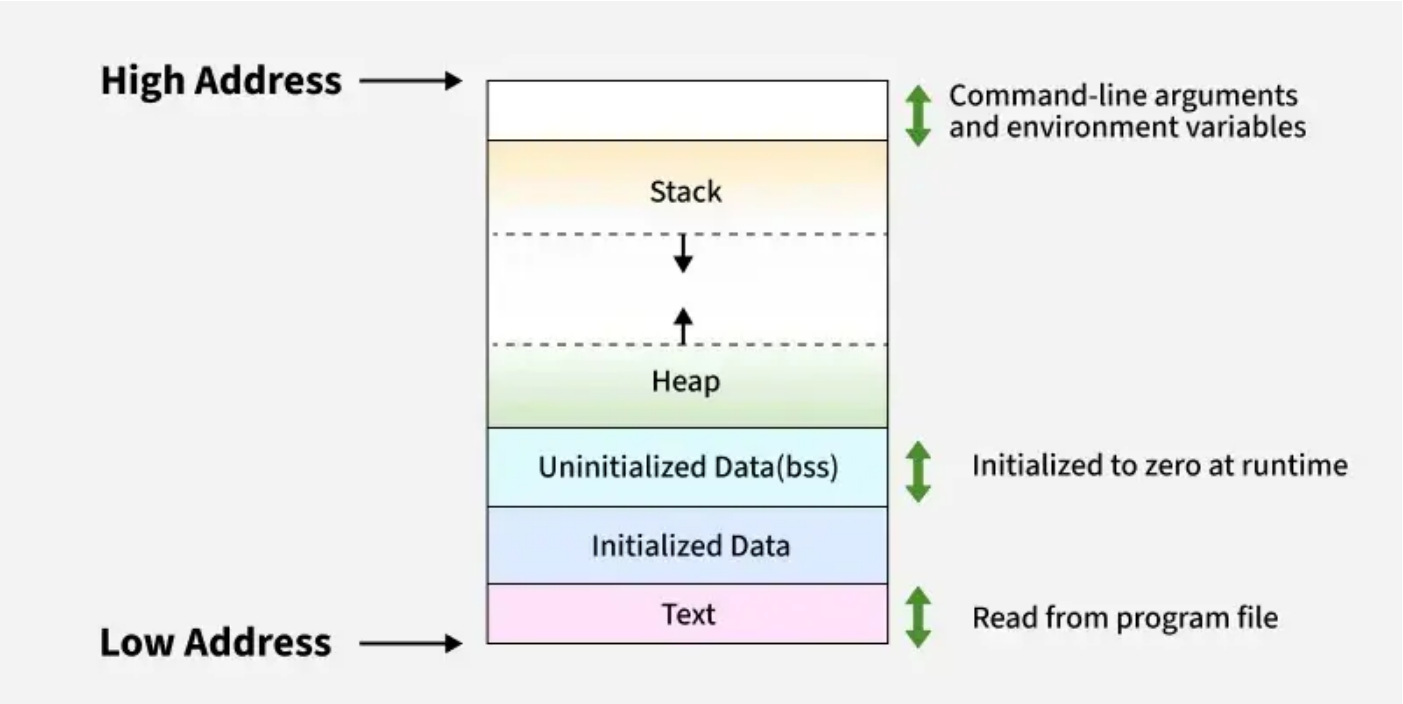
    return 0;
}
```

```
Before setsid:
PID  = 30206
PPID = 7378
PGID = 30206
SID  = 7378
--------------------------------
Child before setsid:
PID  = 30207
PPID = 30206
PGID = 30206
SID  = 7378
--------------------------------
Child after setsid:
PID  = 30207
PPID = 30206
PGID = 30207
SID  = 30207
```

- An overview of sessions, process groups, and processes.



5.Memory Layout in C



When a C program is running, Linux creates a process and assigns it a virtual address space. This is a conceptual model. The actual layout depends on the architecture, kernel, ASLR, linker, loader, etc. Example to view the addresses of its own memory regions:

```
int global_init = 10;
int global_uninit;

static int static_init = 20;
static int static_uninit;

void foo(void)
{
    int local2 = 30;

    printf("foo:             %p\n", (void *)foo);
    printf("local2:           %p\n", (void *)&local2);
}

int main(void)
{
    int local1 = 40;
    int *heap = malloc(sizeof(int));

    *heap = 50;

    printf("main:            %p\n", (void *)main);
```

```
    printf("global_init:     %p\n", (void *)&global_init);
    printf("global_uninit:   %p\n", (void *)&global_uninit);

    printf("static_init:     %p\n", (void *)&static_init);
    printf("static_uninit:   %p\n", (void *)&static_uninit);

    printf("local1:           %p\n", (void *)&local1);

    printf("heap:            %p\n", (void *)heap);

    foo();

    free(heap);

    return 0;
}
```

```
main:            0x60a7608e423b
global_init:     0x60a7608e7010
global_uninit:   0x60a7608e701c
static_init:     0x60a7608e7014
static_uninit:   0x60a7608e7020
local1:           0x7fff5388f93c
heap:            0x60a7654942a0
foo:             0x60a7608e41c9
local2:           0x7fff5388f914
```

You can see this in memory map by using `cat /proc/12345/maps`:

```
60a7608e4000-60a7608e5000 r-xp  .../memory/memory  <--- main, foo
60a7608e7000-60a7608e8000 rw-p  .../memory/memory  <--- global_init, global_uninit,
static_init, static_uninit
60a765494000-60a7654b5000 rw-p  [heap]             <--- heap
7fff53870000-7fff53891000 rw-p  [stack]            <--- local1, local2
```

6.Static Linking vs Dynamic Linking

As we know, when a program is compiled, it undergoes the following steps: source file(.c), preprocesser(.i file), compiler(.s file), assemblier(.o file), linker(executable file).

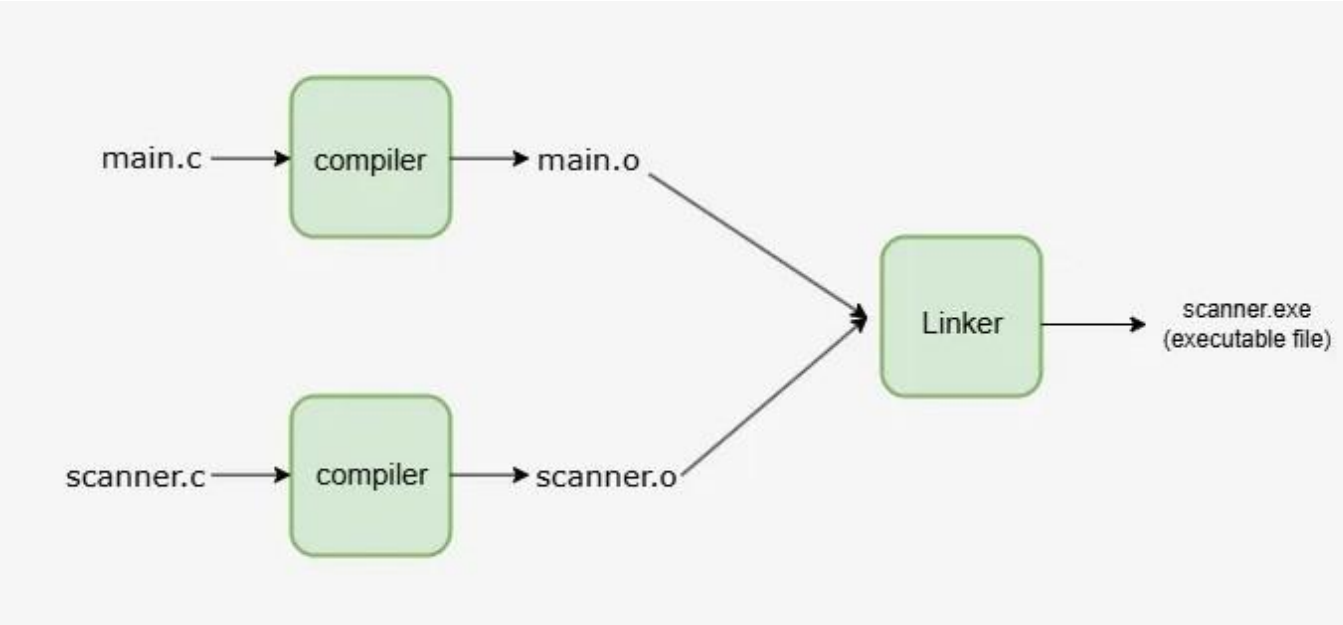
On this page, we will discuss the **Linker** and how it works.
A simple program:

```
#include <stdio.h>


int main(void)
{
    printf("Hello\n");
    return 0;
}
```

We have `printf()` function but it isn't declared in program. It is declared in C library `stdio.h`. The compiler can generate `main.o` with a reference to `printf`. Then, the linker/loader then resolves this reference.

- A definition about **Linker** I found somewhere on the internet:
  'A linker is a special program that combines the object files, generated by the compiler/assembler and other pieces of code to originate an executable file that has a .exe extension.'



Types of Linking:

- **Static Linking**: Copies library code directly into the final executable file during compilation.
- **Dynamic Linking**: Leaves references to libraries, loading them into memory only when the program runs

Some bugs when using dynamic linking:

1. I have two `file linking.c` and `lib_linking.c`:

```
/* linking.c */
#include <stdio.h>

extern int add(int a, int b);

int main(void)
{
    printf("%d", add(10,20));
    return add(10,20);
}
```

```
/* lib_linking.c */
int add(int a, int b)
{
    return a + b;
}
```

I compiled the `lib_linking.c` file into `lib_math.o` and created a dynamic library named `lib_math.so`. Then, I compiled it with `linking.c` using the `gcc linking.c -L. -l_math -o app` command to generate an executable file `app`. Everything went smoothly until I ran the executable; the result was unexpected:

```
./linking_dyna: error while loading shared libraries: lib_math.so: cannot open shared object
file: No such file or directory
```

Why does it happen?
The linker finds the library at build time, but the dynamic loader does not find it at runtime.
To fix this, I need to tell the loader where to look for the library, so I used the command: `export LD_LIBRARY_PATH=.`. It means: look for the current libary here.

```
30
```

2. when changing the data type of a reference in a function from a library

```
/* lib_linking.c */
int add(float a, float b)
{
    return a + b;
}
```

Then run executable file, the result is `0`.

LAB

processes

https://github.com/hungmanh150602/Zhone_Training/tree/main/lab/session_3/zombie

https://github.com/hungmanh150602/Zhone_Training/tree/main/lab/session_3/orphan

race condition between fork and exec

https://github.com/hungmanh150602/Zhone_Training/tree/main/lab/session_3/race_condition

Write a program that prints the addresses of its own memory regions

https://github.com/hungmanh150602/Zhone_Training/tree/main/lab/session_3/memory

Use nm/readelf/ldd/objdump to inspect a self-written binary and produce a short report

https://github.com/hungmanh150602/Zhone_Training/tree/main/lab/session_3/ELF

Compile a multi-argument C function to Assembly with gcc -S and annotate its stack frame: saved %rbp, return address, local variables, and any register-passed arguments

https://github.com/hungmanh150602/Zhone_Training/tree/main/lab/session_3/multi_argument_function

---

Revision #75
Created 2026-08-21 02:24:41 UTC by hung nguyen
Updated 2026-09-28 08:31:58 UTC by hung nguyen